

Programación de servicios y procesos

Unidad 1. Programación multiproceso

2º curso

Desarrollo de Aplicaciones Multiplataforma

Curso 2026/2027

IES Vicente Medina - Archena

Índice de contenidos

1.1 Introducción a los sistemas multitarea	3
Conceptos fundamentales	3
Tipos de multitarea	3
Concurrencia vs. Paralelismo	3
1.2 Procesos: conceptos teóricos	4
¿Qué es un proceso?	4
Diferencia entre programa, ejecutable, proceso, hilo y servicio	4
Estructura de memoria de un proceso	6
Bloque de Control del Proceso (PCB - Process Control Block)	6
Ciclo de Vida y Estados de un Proceso	7
Mecanismos de Creación de Procesos (Linux vs. Windows)	7
Programas, ejecutables, procesos y servicios	8
Computación concurrente y paralela	8
Programación distribuida	8
Hilos (threads)	9
Bifurcación (Fork)	9
1.3 Comunicación y sincronización entre procesos	10
Comunicación entre procesos (IPC - Inter Process Communication)	10
Sincronización entre procesos	11
1.4 Programación de aplicaciones multiplataforma en Java	13
Las Clases Clave del API de Procesos	13
Creación y ejecución de procesos nativos del SO desde JAVA	15
Flujo de Comunicación y Canales E/S	15
Sincronización y control del ciclo de vida	18
Encadenamiento de procesos (Pipelining en JAVA 9+)	22
Control Moderno del Proceso con ProcessHandle (Java 9+)	26
Gestión de errores y excepciones en la invocación de procesos	28
Ejercicio Práctico: Monitor de Diagnóstico y Generador de Logs	32
Enunciado	32
Objetivos del ejercicio	32
Requisitos paso a paso	33
Bibliografía	34

Unidad 1

Programación multiproceso

1.1 Introducción a los sistemas multitarea

Conceptos fundamentales

- **Sistemas monotarea** → Solo pueden ejecutar un programa a la vez. El procesador está completamente dedicado a esa única tarea hasta que finaliza.
- **Sistemas multitarea** → Permiten que varios programas o procesos se ejecuten de forma simultánea (o parezcan ejecutarse al mismo tiempo) compartiendo los recursos del sistema (CPU, memoria, E/S).

Tipos de multitarea

- **Multitarea cooperativa**
 - El propio programa/proceso decide cuándo ceder el control de la CPU a otros programas.
 - *Inconveniente*: Si un programa entra en un bucle infinito o se "cuelga", bloquea todo el sistema operativo.
- **Multitarea apropiativa**
 - El **Sistema Operativo (SO)** tiene el control total y puede suspender un proceso en cualquier momento para concederle tiempo de CPU a otro.
 - Utiliza un componente del SO llamado **Planificador (Scheduler)**, que asigna rodajas o cuantos de tiempo (*time slots*) a cada proceso.
 - Es el estándar en los SO modernos (Linux, Windows 10/11, macOS).

Concurrencia vs. Paralelismo

Es muy común confundir estos dos términos, pero representan ideas distintas a nivel de hardware:

- **Concurrencia (intercalado de tareas)**
 - Ocurre cuando varias tareas avanzan en el mismo periodo de tiempo, pero **en un único núcleo de CPU**.
 - El procesador conmuta tan rápido entre un proceso y otro (*conmutación de contexto*) que crea la **ilusión** de ejecución simultánea.
- **Paralelismo (Ejecución simultánea real)**
 - Ocurre cuando dos o más tareas se ejecutan **físicamente al mismo instante de tiempo**.
 - Requiere **múltiples núcleos de procesador (Multicore)** o múltiples procesadores físicos.

1.2 Procesos: conceptos teóricos

¿Qué es un proceso?

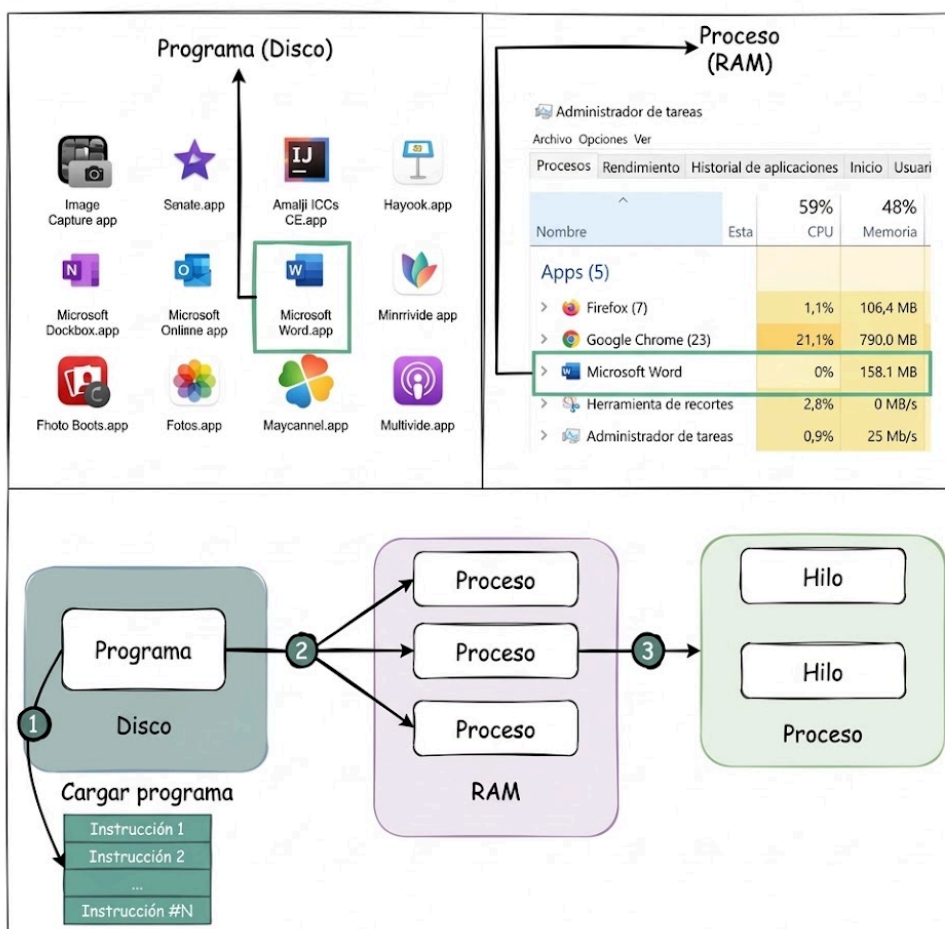
- **Programa** → Entidad *estática*. Es un conjunto de instrucciones almacenadas en un disco (un ejecutable `.exe`, `.bin`, `.jar`).
- **Proceso** → Entidad *dinámica*. Es un programa en ejecución, compuesto por el código ejecutable, sus datos, su pila, su contador de programa y los recursos asignados por el SO.

Diferencia entre programa, ejecutable, proceso, hilo y servicio

En la programación multiproceso es fundamental diferenciar la entidad que reside en disco de las unidades de ejecución que el sistema operativo gestiona en memoria RAM.

Programa vs Proceso vs Hilo

 blog.bytbytego.com

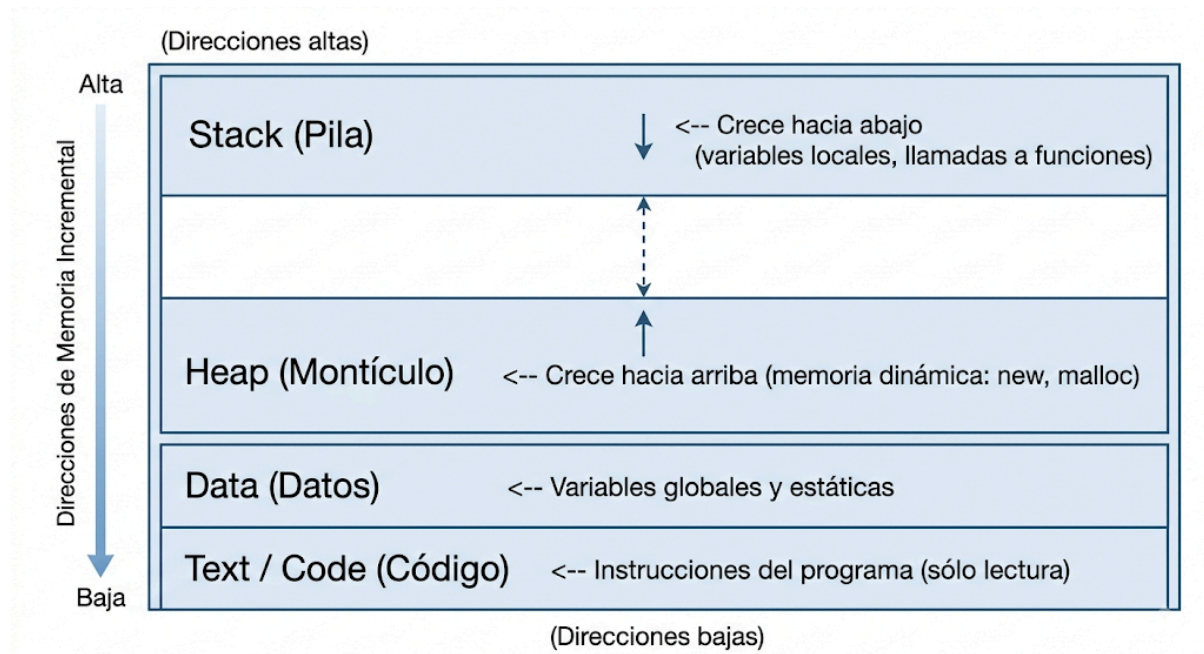


- **Programa:** Conjunto de instrucciones lógicas y secuencias de código escritas por el programador. Es una **entidad pasiva** almacenada como código fuente o archivo estructurado.
- **Ejecutable:** Archivo binario generado tras compilar o empaquetar el programa (ej. `.exe` en Windows, archivo con permisos `+x` en Linux/ELF, o un `.jar`). Permanece almacenado en el disco duro a la espera de ser invocado.
- **Proceso:** Instancia de un ejecutable cargado en la memoria RAM y en estado de ejecución. Es una **entidad activa** a la que el sistema operativo asigna un identificador único (PID), un espacio de memoria propio (código, datos, *heap*, *stack*) y recursos del sistema (descriptores de archivos, *sockets*).
- **Hilo (Thread):** Unidad mínima de ejecución dentro de un proceso. Varios hilos dentro de un mismo proceso comparten el espacio de memoria (variables globales, *heap*) y los recursos asignados, pero cada uno mantiene su propio contador de programa y pila de llamadas.
- **Servicio (Daemon en Linux):** Proceso especial diseñado para ejecutarse en segundo plano (*background*) de forma continua, sin requerir una interfaz de usuario directa ni la interacción con un operador. Se inicia habitualmente con el sistema operativo y atiende peticiones (ej. un servidor web como `nginx` o un gestor de base de datos como `mysqld`).

Concepto	Estado / Ubicación	Espacio de Memoria	Interfaz de Usuario	aislamiento
Programa	Pasivo (Código fuente)	N/A (Solo ocupa disco)	No aplica	No aplica
Ejecutable	Pasivo (Disco duro)	N/A (Solo ocupa disco)	No aplica	No aplica
Proceso	Activo (Memoria RAM)	Propio e aislado	Opcional (GUI/CLI)	Alto (aislado por el SO)
Hilo	Activo (Dentro del proceso)	Compartido dentro del proceso	Depende del proceso contenedor	Bajo (comparten memoria)
Servicio	Activo (Segundo plano en RAM)	Propio de su proceso	Sin interfaz directa	Alto (modo demonio)

Estructura de memoria de un proceso

Cuando el Sistema Operativo carga un proceso en memoria principal (RAM), le asigna un espacio de direccionamiento privado dividido en 4 secciones principales:



- **Texto/Código** → Contiene el código máquina ejecutable.
- **Datos** → Almacena variables globales y variables estáticas inicializadas.
- **Heap (Montículo)** → Memoria reservada dinámicamente en tiempo de ejecución (ej. mediante `malloc()` en C o `new` en Java).
- **Stack (Pila)**: Almacena argumentos de funciones, variables locales y direcciones de retorno tras llamadas a métodos.

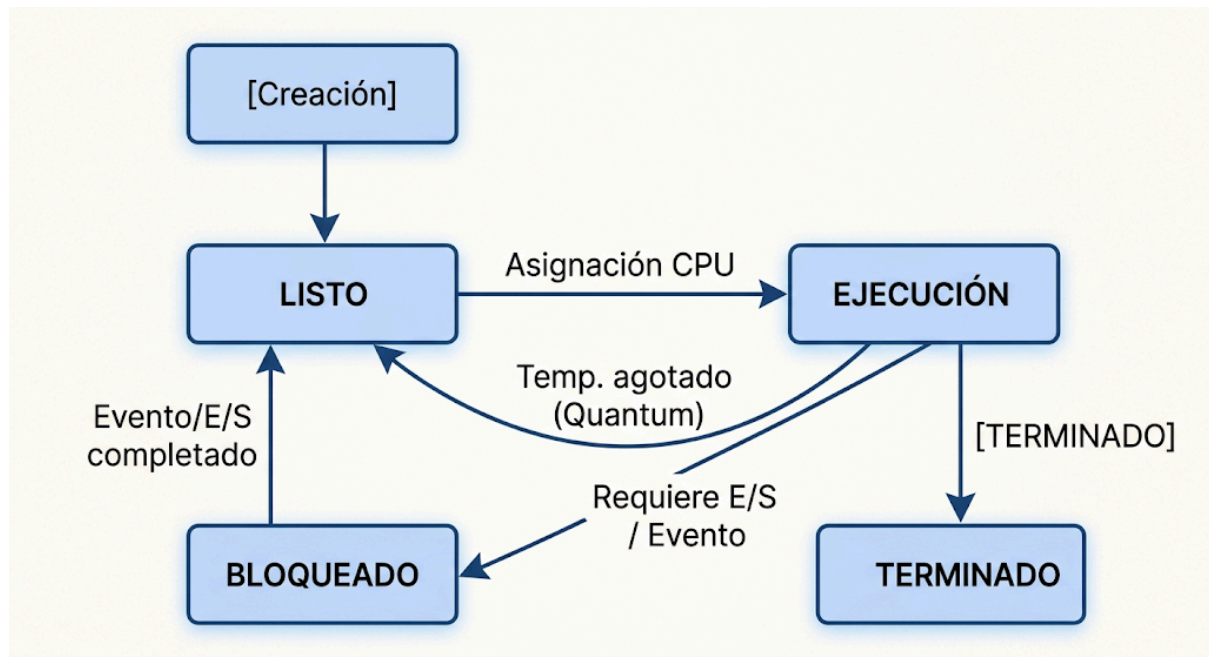
Bloque de Control del Proceso (PCB - *Process Control Block*)

El **PCB** es una estructura de datos interna del Sistema Operativo que guarda toda la información necesaria para gestionar un proceso específico. Cuando el SO conmuta de un proceso a otro (*conmutación de contexto*), guarda el estado en su PCB.

Contenido clave del PCB:

1. **PID (*Process Identifier*)**: Identificador único del proceso en el sistema.
2. **PPID (*Parent PID*)**: PID del proceso padre que lo creó.
3. **Estado del proceso**: (Nuevo, Listo, Ejecución, Bloqueado, Terminado).
4. **Contador de Programa (PC)**: Dirección de la siguiente instrucción a ejecutar.
5. **Registros de CPU**: Copia del contenido de los registros del procesador para reanudar el proceso exactamente donde se quedó.
6. **Información de gestión de memoria**: Tablas de páginas o segmentos asociados.
7. **Contabilidad e I/O**: Archivos abiertos, sockets de red asociados, tiempo de CPU usado, etc.

Ciclo de Vida y Estados de un Proceso



1. **Nuevo / Creando:** El proceso se está creando en memoria.
2. **Listo (Ready):** El proceso está cargado en memoria y esperando a que el *scheduler* le asigne tiempo de CPU.
3. **Ejecución (Running):** El proceso tiene el control de la CPU y sus instrucciones se están ejecutando.
4. **Bloqueado / Esperando (Waiting/Blocked):** El proceso no puede continuar hasta que ocurra un evento externo (ej. lectura de un archivo en disco, entrada de teclado, recepción de datos por red).
5. **Terminado / Finalizado (Terminated):** Ha completado su ejecución o ha sido abortado. Libera sus recursos.

Mecanismos de Creación de Procesos (Linux vs. Windows)

Los procesos se crean a partir de otros procesos existentes, formando un **árbol de procesos**:

- **Jerarquía POSIX / Linux:**
 - **fork():** Duplica el proceso actual (padre) creando un proceso hijo idéntico en memoria (con un PID distinto).
 - **exec():** Reemplaza la imagen de memoria del proceso actual por un nuevo programa ejecutable.
 - *Pauta habitual:* **fork()** para crear al hijo + **exec()** para hacer que ejecute otro programa.
- **Windows:**
 - Utiliza la API **CreateProcess()**, que combina en un solo paso la creación del proceso y la carga del nuevo código ejecutable.

Programas, ejecutables, procesos y servicios

- **Programa:** Archivo estático que contiene código fuente escrito por el desarrollador (ej. `Main.java`, `script.py`).
- **Ejecutable:** Programa traducido a lenguaje máquina o bytecode listo para ser cargado por el SO (ej. `app.exe`, `script.sh`, `Main.class`).
- **Proceso:** Instancia activa en memoria de un ejecutable, con un PID asignado y recursos propios del SO (interfaz, CPU, memoria).
- **Servicio (o Demonio):** Proceso especial que se ejecuta en segundo plano (*background*) sin interfaz de usuario interactiva y que suele iniciarse con el sistema (ej. `httpd`, `mysqld`, servicios de Windows).

Ejemplo práctico:

- Un archivo `mysql.exe` en tu disco duro es el **ejecutable**.
- Cuando se inicia al arrancar el ordenador y gestiona la base de datos sin mostrar ninguna ventana, es un **servicio** ejecutándose como un **proceso** del sistema.

Computación concurrente y paralela

- **Computación Concurrente:** Diseño de software que gestiona múltiples tareas avanzando en periodos de tiempo solapados. Se puede dar con **un solo núcleo** mediante *conmutación de contexto* (tiempo compartido).
- **Computación Paralela:** Ejecución simultánea real de dos o más tareas en el **mismo instante de tiempo**. Requiere **hardware multinúcleo**.

Programación distribuida

Es un modelo de arquitectura donde componentes de software ubicados en **distintos ordenadores conectados en red** (nodos) se comunican y coordinan sus acciones mediante el intercambio de mensajes (ej. usando Sockets, APIs REST o gRPC).

- **Diferencia con multiproceso local:** En multiproceso local, los procesos están en el mismo chip/placa y comparten el SO; en distribuida, cada nodo tiene su propia memoria y su propio sistema operativo.

Ejemplo: Una consulta en Google Search: tu navegador envía una petición y cientos de servidores distribuidos por el mundo procesan fragmentos del índice simultáneamente para devolverte el resultado.

Hilos (threads)

Un hilo es la unidad más pequeña de ejecución que puede ser programada por el Sistema Operativo. Un único proceso puede albergar múltiples hilos que se ejecutan de forma concurrente.

- **Proceso vs. Hilo:**
 - Los **procesos** están aislados y **NO** comparten memoria por defecto.
 - Los **hilos** del mismo proceso **Sí comparten** el espacio de memoria (Heap, Variables globales y Código), pero cada uno tiene su propio conjunto de registros y su propia **Stack (Pila)**.

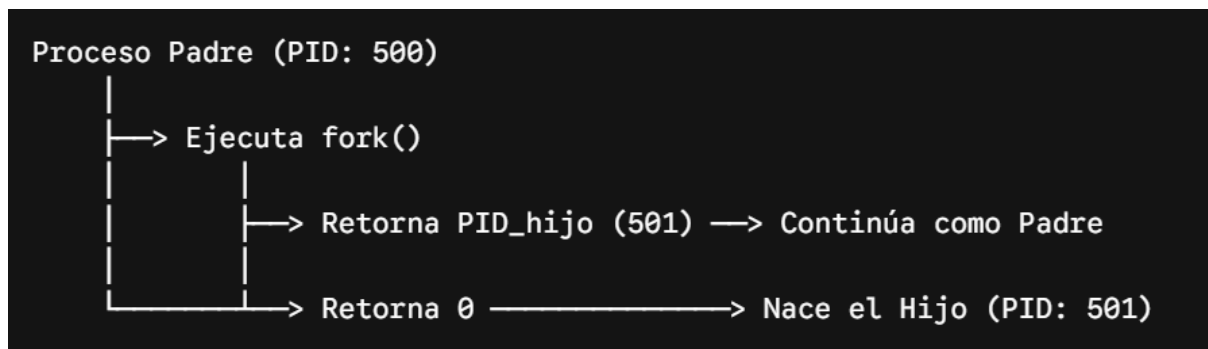


Ejemplo: En un procesador de texto (como Word), un hilo atiende al teclado mientras otro verifica la ortografía en segundo plano y otro guarda cambios automáticamente en el disco.

Bifurcación (*Fork*)

Es una llamada al sistema operativo (típica en sistemas POSIX/Linux mediante la función `fork()`) que permite a un proceso en ejecución (**padre**) crear una copia exacta de sí mismo (**hijo**).

- **Comportamiento clave:**
 - El hijo hereda una copia de la memoria del padre y los mismos descriptores de archivos abiertos.
 - A partir del punto de bifurcación, ambos procesos continúan su ejecución de forma independiente.
 - La función `fork()` devuelve **0** al proceso hijo y el **PID del hijo** al proceso padre.



Ejemplo (Pseudocódigo C):

```
C/C++
int pid = fork();

if (pid == 0) {
    // Código que ejecuta SOLO el hijo
    printf("Soy el hijo con PID: %d\n", getpid());
} else {
    // Código que ejecuta SOLO el padre
    printf("Soy el padre, he creado al hijo %d\n", pid);
}
```

1.3 Comunicación y sincronización entre procesos

Comunicación entre procesos (IPC - *Inter Process Communication*)

Como los procesos tienen **espacios de memoria totalmente aislados**, el Sistema Operativo debe ofrecer mecanismos explícitos para que puedan intercambiar información.

Tuberías (*Pipes*):

- **Tuberías anónimas:** Canales unidireccionales (FIFO) para procesos emparentados (padre-hijo). Un proceso escribe en un extremo y el otro lee del opuesto.
- **Tuberías con nombre (*FIFOs*):** Persisten en el sistema de archivos y permiten la comunicación entre procesos independientes (no emparentados).

Memoria Compartida (*Shared Memory*):

- El SO mapea una región de la memoria física en el espacio de direcciones de múltiples procesos.
- **Ventaja:** Es el método IPC más rápido (sin copia de datos por el kernel).

- **Desventaja:** Requiere mecanismos de sincronización explícitos para evitar conflictos de lectura/escritura.

Colas de Mensajes (*Message Queues*):

- El SO gestiona una lista enlazada de mensajes almacenada en el kernel. Los procesos leen y escriben bloques de datos estructurados de forma asíncrona.

Sockets:

- Mecanismo bidireccional basado en endpoints de red. Funciona tanto para procesos en la **misma máquina** (sockets UNIX / IPC local) como entre máquinas distintas a través de red (TCP/UDP).

Señales (*Signals*):

- Notificaciones de interrupción de software enviadas por el SO o un proceso a otro para avisar de un evento (ej. **SIGINT** para interrumpir, **SIGKILL** para forzar el cierre).

Mecanismo IPC	Ámbito	Velocidad	Requiere Sincronización Explícita
Memoria Compartida	Local	Muy Alta	Sí
Pipes / FIFOs	Local	Alta	No (gestión interna del SO)
Colas de Mensajes	Local	Media-Alta	No (gestión interna del SO)
Sockets	Local / Red	Media	No (gestión interna del SO)

Sincronización entre procesos

Cuando dos o más procesos acceden de forma concurrente a recursos compartidos (ficheros, memoria compartida, hardware), se deben coordinar para mantener la integridad de los datos.

Problemas Fundamentales:

- **Condición de Carrera (*Race Condition*):** Situación en la que el resultado final depende del orden impredecible de ejecución de los procesos.

Ejemplo: Dos procesos leen el saldo bancario (100€) a la vez e intentan ingresar 50€. Si ambos leen la variable original antes de guardar, el resultado final será 150€ en lugar de 200€.

Solución: Garantizar la exclusión mutua en la sección crítica.

- **Sección Crítica (*Critical Section*):** Fragmento de código donde un proceso accede a un recurso compartido. **Solo un proceso debe ejecutar su sección crítica a la vez.**

Ejemplo: La línea concreta de código donde se ejecuta `saldo = saldo + ingreso`.

Solución: Proteger el bloque usando cerrojos (*locks*), palabras clave de sincronización (`synchronized`) o semáforos.

- **Interbloqueo (*Deadlock*):** Situación donde dos o más procesos quedan bloqueados indefinidamente esperando un recurso que posee el otro.

Ejemplo: El Proceso A retiene el Recurso 1 y solicita el Recurso 2. Al mismo tiempo, el Proceso B retiene el Recurso 2 y solicita el Recurso 1. Ambos quedan pausados para siempre.

Solución: Definir un orden estricto de adquisición de recursos o establecer tiempos límite (*timeouts*).

- **Inanición (*Starvation*):** Un proceso nunca llega a ejecutarse porque el planificador asigna siempre la CPU o el recurso a otros procesos.

Ejemplo: Un proceso con prioridad muy baja nunca recibe tiempo de CPU porque el planificador da paso continuamente a procesos de prioridad alta.

Solución: Implementar políticas de reparto equitativo (*fairness*) o envejecimiento (*aging*), incrementando la prioridad del proceso a medida que aumenta su tiempo en espera.

Mecanismos de sincronización (Primitivas)

Exclusión Mutua (*Mutex* / *Cerrojo*):

- Objeto binario que actúa como un cerrojo. Un proceso hace `lock()` antes de entrar a la sección crítica y `unlock()` al salir. Si otro proceso intenta entrar, queda bloqueado.

Semáforos (*Dijkstra*):

- Estructura que mantiene una variable entera contador para controlar el acceso a recursos limitados:
 - `wait()` / `P()`: Decrementa el contador. Si llega a < 0 , el proceso se bloquea.
 - `signal()` / `V()`: Incrementa el contador y despierta a un proceso bloqueado.
- *Semáforo binario*: Solo toma valores 0 y 1 (actúa como un *Mutex*).

- **Semáforo contador:** Permite el acceso simultáneo a un número máximo N de procesos.

1.4 Programación de aplicaciones multiplataforma en Java

Las Clases Clave del API de Procesos

Clase	Responsabilidad Principal	Métodos Clave	Descripción
<code>ProcessBuilder</code>	Configurar y lanzar nuevos procesos.	<code>command()</code>	Define o modifica el comando del sistema operativo y sus argumentos que se van a ejecutar. Puede recibir una lista de cadenas o argumentos independientes.
		<code>directory()</code>	Establece el directorio de trabajo desde el cual el subprocesso iniciará su ejecución (equivale al comando <code>cd</code> antes de ejecutar).
		<code>redirectOutput()</code>	Redirige la salida estándar (<code>STDOUT</code>) del proceso a un destino externo (un archivo con <code>Redirect.to(file)</code> , la consola de Java con <code>Redirect.INHERIT</code> o descartarla).
		<code>start()</code>	Crea e inicia el subprocesso nativo con la configuración especificada. Devuelve un objeto de tipo <code>Process</code> para interactuar con él.
<code>Process</code>	Representar un proceso en ejecución y controlar su flujo.	<code>getInputStream()</code>	Obtiene el flujo de entrada (InputStream) para leer la salida estándar (STDOUT) del subprocesso.

		<code>getOutputStream()</code>	Obtiene el flujo de escritura (<i>OutputStream</i>) conectado a la entrada estándar del hijo. Permite enviar datos o simular que el usuario teclea algo dentro del subproceso.
		<code>waitFor()</code>	Bloquea el hilo ejecutor en Java hasta que el subproceso finaliza. Devuelve un número entero correspondiente al código de salida del sistema (habitualmente <code>0</code> indica ejecución correcta).
		<code>destroy()</code>	Solicita la terminación del proceso de forma ordenada y limpia (equivale a enviar una señal <code>SIGTERM</code> en Linux/macOS).
<code>ProcessHandle</code> (Java 9+)	Inspeccionar metadatos nativos del sistema (PID, usuario, tiempo CPU).	<code>current()</code>	Método estático que devuelve un objeto <code>ProcessHandle</code> referente al propio proceso Java que se está ejecutando actualmente.
		<code>pid()</code>	Devuelve el identificador único de proceso (PID) asignado directamente por el sistema operativo como un valor de tipo <code>long</code> .
		<code>info()</code>	Retorna una captura de metadatos del proceso (<code>ProcessHandle.Info</code>), como el usuario ejecutor, el comando exacto de inicio, la hora de arranque y el tiempo consumido de CPU.
		<code>allProcesses()</code>	Devuelve un objeto de tipo <code>Stream<ProcessHandle></code> con todos los procesos activos y visibles en el sistema operativo en ese instante.

Creación y ejecución de procesos nativos del SO desde JAVA

Flujo de Comunicación y Canales E/S

La comunicación entre el proceso **Padre** (tu programa Java) y el proceso **Hijo** (el ejecutable lanzado) se realiza mediante tuberías (*streams*).

Los nombres de los métodos en Java se leen desde el punto de vista del **Padre**:

- `getInputStream()`: Lee la **salida estándar (STDOUT)** del hijo.
- `getErrorStream()`: Lee los **errores (STDERR)** del hijo.
- `getOutputStream()`: Escribe datos en la **entrada estándar (STDIN)** del hijo.

Ejemplo 1: Crear un Proceso y Leer su Salida

Este código lanza un comando del sistema de forma multiplataforma y captura su respuesta línea a línea.

```
Java
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.List;

public class LectorProceso {
    public static void main(String[] args) {
        // Detectar SO para elegir el comando adecuado
        boolean esWindows =
System.getProperty("os.name").toLowerCase().contains("win");
        List<String> comando = esWindows
            ? List.of("cmd.exe", "/c", "dir")
            : List.of("ls", "-la");

        // 1. Configurar el proceso
        ProcessBuilder pb = new ProcessBuilder(comando);

        try {
            // 2. Iniciar el proceso
            Process proceso = pb.start();

            // 3. Capturar la salida del hijo (getInputStream)
            try (BufferedReader reader = new BufferedReader(new
InputStreamReader(proceso.getInputStream()))) {
                String linea;
                while ((linea = reader.readLine()) != null) {
```

```
        System.out.println("[SALIDA HIJO] " + linea);
    }
}

// 4. Esperar a que el proceso termine y obtener código de salida
(0 = Éxito)
int codigoSalida = proceso.waitFor();
System.out.println("Proceso finalizado con código: " +
codigoSalida);

} catch (Exception e) {
    e.printStackTrace();
}
}
}
```

Ejemplo 2: Redirección de Entrada y Salida Directa

Si no necesitas procesar la salida en Java y solo quieres que el proceso hijo se muestre directamente en la consola del ordenador o redirigirlo a un fichero:

```
Java
import java.io.File;
import java.io.IOException;

public class RedireccionProceso {
    public static void main(String[] args) throws IOException,
InterruptedException {
        ProcessBuilder pb = new ProcessBuilder("ping", "127.0.0.1");

        // Opción A: Heredar la consola del Padre (se imprime directamente en
pantalla)
        pb.inheritIO();

        // Opción B: Redirigir la salida directamente a un archivo en disco
        // File log = new File("salida_ping.txt");
        // pb.redirectOutput(log);

        Process p = pb.start();
        p.waitFor();
    }
}
```

Ejemplo 3: Comunicación Bidireccional (Enviar datos al Hijo)

Para pasarle datos a la entrada estándar de un proceso secundario (simular que el usuario escribe por teclado):

```
Java
import java.io.OutputStream;
import java.io.PrintWriter;
import java.util.Scanner;

public class InteraccionProceso {
    public static void main(String[] args) throws Exception {
        // Ejemplo: Lanzar el intérprete de Python o un subprocesso interactivo
        ProcessBuilder pb = new ProcessBuilder("python", "-u", "-c",
            "import sys; nombre = sys.stdin.readline(); print('Hola ' +
nombre)");

        Process p = pb.start();

        // 1. Escribir en la entrada del hijo usando getOutputStream()
        try (PrintWriter writer = new PrintWriter(p.getOutputStream())) {
            writer.println("Carlos"); // Simulamos que el usuario escribe
"Carlos" + Enter
            writer.flush(); // Forzamos el envío inmediato
        }

        // 2. Leer la respuesta del hijo
        try (Scanner scanner = new Scanner(p.getInputStream())) {
            if (scanner.hasNextLine()) {
                System.out.println("Respuesta recibida: " +
scanner.nextLine());
            }
        }

        p.waitFor();
    }
}
```

Sincronización y control del ciclo de vida

En la programación multiproceso, lanzar un subprocesso es una operación **asíncrona**. Esto significa que cuando el proceso padre ejecuta `ProcessBuilder.start()`, el sistema operativo crea la nueva entidad en memoria y devuelve el control al programa Java de inmediato, continuando la ejecución de las líneas siguientes mientras el hijo se ejecuta en paralelo.

Para desarrollar aplicaciones robustas y coordinadas, el proceso padre debe disponer de mecanismos explícitos para esperar la finalización del hijo, controlar los tiempos de

ejecución máximos (*timeouts*) y responder ante bloqueos o terminaciones no deseadas.

Espera activa vs. Espera bloqueante (waitFor)

El API de Java ofrece dos enfoques principales para gestionar el ciclo de vida del subprocesso desde el proceso padre:

- **waitFor() (Espera bloqueante):** Detiene la ejecución del hilo actual de Java hasta que el subprocesso nativo finaliza su ejecución y devuelve su código de salida (*exit code*).
- **waitFor(long timeout, TimeUnit unit) (Espera con límite de tiempo):** Introduce un tiempo máximo de espera para evitar bloqueos indefinidos si el hijo se cuelga o entra en un bucle infinito. Devuelve un valor booleano (**true** si el proceso terminó dentro del plazo, **false** si se agotó el tiempo).

```
Java
Process proceso = new ProcessBuilder("ping", "-c", "5", "127.0.0.1").start();

// Esperar con un tiempo límite de 3 segundos
boolean finalizado = proceso.waitFor(3, TimeUnit.SECONDS);

if (!finalizado) {
    System.out.println("El proceso tardó demasiado. Cancelando...");
    proceso.destroy(); // Solicitar terminación limpia
} else {
    System.out.println("Proceso completado con código: " +
        proceso.exitValue());
}
```

Destrucción Limpia vs. Forzada (destroy vs. destroyForcibly)

Cuando un subprocesso debe interrumpirse antes de su conclusión natural, el API de Java ofrece dos métodos principales de finalización:

Método	Señal OS (Linux/macOS)	Comportamiento OS (Windows)	Descripción
--------	---------------------------	--------------------------------	-------------

<code>destroy()</code>	SIGTERM (15)	TerminateProcess	Solicitud de terminación ordenada. Permite al proceso liberar recursos o guardar estado antes de cerrarse.
<code>destroyForcibly()</code>	SIGKILL (9)	TerminateProcess (inmediato)	Terminación abrupta e inmediata del proceso por el Kernel. El proceso no puede capturar la señal ni limpiar recursos.

Control Asíncrono con CompletableFuture (onExit)

Desde **Java 9**, la clase `Process` introduce el método `onExit()`, el cual devuelve un `CompletableFuture<Process>`. Esto permite registrar *callbacks* asíncronos que se ejecutarán automáticamente en el instante en que el subprocesso finalice, sin necesidad de bloquear el hilo principal de la aplicación.

```
Java
ProcessBuilder pb = new ProcessBuilder("ls", "-la");
Process proceso = pb.start();

// Configurar acción asíncrona al terminar el proceso
proceso.onExit().thenAccept(p -> {
    System.out.println("El proceso " + p.pid() + " ha terminado con código " +
        p.exitValue());
});

System.out.println("El hilo principal de Java continúa realizando otras
tareas...");
```

Códigos de Salida (Exit Values / Exit Status)

Por convención estándar en sistemas operativos, el código de salida devuelto por un proceso indica el resultado de su ejecución:

- **0:** Terminación correcta (Éxito).
- **Distinto de 0 (ej. 1, 2, 127, 137):** Indica que se produjo un error específico o que el proceso fue destruido por una señal externa del sistema.

Ejemplo Completo de Código Java

```
Java
import java.io.IOException;
import java.util.concurrent.TimeUnit;

public class ControlCicloVidaProceso {

    public static void main(String[] args) {
        boolean esWindows =
            System.getProperty("os.name").toLowerCase().contains("win");

        // Comando que simula un trabajo largo (10 segundos)
        ProcessBuilder pb = esWindows
            ? new ProcessBuilder("cmd.exe", "/c", "timeout /t 10 /nobreak")
            : new ProcessBuilder("sleep", "10");

        try {
            System.out.println("=== INICIANDO SUBPROCESO ===");
            Process proceso = pb.start();
            long pid = proceso.pid();
            System.out.println("Proceso iniciado con PID: " + pid);

            // 1. Monitoreo asíncrono mediante onExit() (Java 9+)
            proceso.onExit().thenAccept(p ->
                System.out.println("[Callback Asíncrono] El proceso PID " +
                    p.pid() + " ha finalizado."));

            // 2. Espera con Timeout (esperamos máximo 3 segundos)
            System.out.println("Esperando finalización (máximo 3 segundos)...");
            boolean terminadoApertura = proceso.waitFor(3, TimeUnit.SECONDS);

            // 3. Control del estado si no termina a tiempo
            if (!terminadoApertura) {
                System.out.println("ATENCIÓN: El proceso ha superado el límite de tiempo de 3s.");
            }
        }
    }
}
```

```
        System.out.println("Enviando orden de terminación limpia  
(destroy)...");  
        proceso.destroy();  
  
        // Damos un margen de 2 segundos para la terminación limpia  
        if (!proceso.waitFor(2, TimeUnit.SECONDS)) {  
            System.out.println("El proceso no responde. Forzando  
terminación abrupta (destroyForcibly)...");  
            proceso.destroyForcibly();  
        }  
    }  
  
    // 4. Espera final y lectura del código de salida  
    int codigoSalida = proceso.waitFor();  
    System.out.println("\n=== RESUMEN DE EJECUCIÓN ===");  
    System.out.println("PID: " + pid);  
    System.out.println("¿Está vivo?: " + proceso.isAlive());  
    System.out.println("Código de salida final: " + codigoSalida);  
  
    } catch (IOException e) {  
        System.err.println("Error de E/S al lanzar el proceso: " +  
e.getMessage());  
    } catch (InterruptedException e) {  
        System.err.println("El hilo principal fue interrumpido: " +  
e.getMessage());  
        Thread.currentThread().interrupt();  
    }  
}  
}
```

Encadenamiento de procesos (*Pipelining* en JAVA 9+)

El **encadenamiento de procesos** (*pipelining*) consiste en conectar la salida estándar (*STDOUT*) de un proceso directamente a la entrada estándar (*STDIN*) del siguiente proceso. De esta forma, el segundo proceso procesa los datos generados por el primero a medida que se van produciendo, emulando el operador tubería `|` (*pipe*) nativo de las consolas de comandos (Bash, PowerShell, CMD).

Antes de Java 9, esto requería crear manualmente varios subprocesos, obtener sus flujos E/S (`getInputStream()` y `getOutputStream()`) y crear hilos secundarios para copiar manualmente los bytes de un flujo a otro, lo cual era complejo y propenso a bloqueos (*deadlocks*).

El método `ProcessBuilder.startPipeline()`

A partir de **Java 9**, la API introduce el método estático `ProcessBuilder.startPipeline(List<ProcessBuilder> builders)`, que automatiza completamente la creación y conexión de tuberías entre procesos a nivel de Kernel del sistema operativo.

Java

```
public static List<Process> startPipeline(List<ProcessBuilder> builders) throws  
IOException
```

Características principales:

1. **Redirección automática:** La salida estándar del proceso N se reconecta de manera transparente a la entrada estándar del proceso N+1.
2. **Optimización nativa:** El intercambio de datos entre procesos se realiza a nivel de sistema operativo sin pasar por la memoria de la Máquina Virtual de Java (JVM), reduciendo el consumo de CPU y memoria.
3. **Control individual:** El método devuelve un `List<Process>` donde cada elemento representa uno de los procesos del pipeline en el orden en que fueron pasados, permitiendo monitorear o esperar a cualquier proceso de la cadena de forma independiente.

Manejo de E/S en los extremos del Pipeline

Es muy importante tener en cuenta cómo se gestionan las entradas y salidas en los procesos que forman la cadena:

- **Procesos intermedios:** No se debe modificar manualmente su `redirectInput()` ni `redirectOutput()`, ya que Java los configura automáticamente para conectarse con sus procesos adyacentes.
- **Primer proceso (Origen):** Su entrada estándar se mantiene en `Redirect.PIPE` (por defecto) o puede redirigirse desde un archivo (`Redirect.from(File)`).
- **Último proceso (Destino):** Su salida estándar puede leerse desde Java mediante `getInputStream()` o redirigirse a un archivo/consola (`Redirect.to(File)` / `Redirect.INHERIT`).

Ejemplo 1: Tubería básica equivalente a comando de Terminal

El siguiente ejemplo simula el comando de terminal

```
cat archivo.txt | grep "ERROR" | wc -l:
```

```
Java
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.Arrays;
import java.util.List;

public class PipelineBasico {
    public static void main(String[] args) {
        try {
            // 1. Definir los procesos de la tubería
            ProcessBuilder pb1 = new ProcessBuilder("cat", "logs.txt");
            ProcessBuilder pb2 = new ProcessBuilder("grep", "ERROR");
            ProcessBuilder pb3 = new ProcessBuilder("wc", "-l");

            // 2. Iniciar el pipeline completo
            List<Process> procesos = ProcessBuilder.startPipeline(
                Arrays.asList(pb1, pb2, pb3)
            );

            // 3. Obtener el último proceso de la cadena para leer su resultado
            Process ultimoProceso = procesos.get(procesos.size() - 1);

            try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(ultimoProceso.getInputStream()))) {

                String linea;
                while ((linea = reader.readLine()) != null) {
                    System.out.println("Número de líneas con ERROR: " +
linea.trim());
                }
            }

            // 4. Esperar a que todos los procesos terminen correctamente
            for (Process p : procesos) {
                p.waitFor();
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Ejemplo 2: Pipeline con redirección de entrada/salida a ficheros

También es posible combinar `startPipeline()` con redirección a ficheros para procesar información sin cargarla en memoria Java:

```
Java
import java.io.File;
import java.util.Arrays;
import java.util.List;

public class PipelineArchivos {
    public static void main(String[] args) {
        try {
            File entrada = new File("datos_brutos.txt");
            File salida = new File("datos_ordenados.txt");

            // Proceso 1: Leer desde un archivo de entrada
            ProcessBuilder pb1 = new
ProcessBuilder("cat").redirectInput(entrada);

            // Proceso 2: Filtrar filas que contengan "2024"
            ProcessBuilder pb2 = new ProcessBuilder("grep", "2024");

            // Proceso 3: Ordenar resultados y guardarlos directamente en un
archivo
            ProcessBuilder pb3 = new
ProcessBuilder("sort").redirectOutput(salida);

            // Arrancar el pipeline
            List<Process> pipeline =
ProcessBuilder.startPipeline(Arrays.asList(pb1, pb2, pb3));

            // Esperar al último proceso
            Process ultimo = pipeline.get(pipeline.size() - 1);
            int exitCode = ultimo.waitFor();

            if (exitCode == 0) {
                System.out.println("Pipeline ejecutado con éxito. Archivo
generado en 'datos_ordenados.txt'");
            } else {
                System.err.println("El pipeline falló con código de salida: " +
exitCode);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
}  
}
```

Consideraciones prácticas y buenas prácticas

- **Sistemas Operativos:** Los comandos pasados a los `ProcessBuilder` en un pipeline dependen del sistema operativo donde se ejecute la JVM (por ejemplo, comandos de Linux/macOS como `cat`, `grep` no están de forma predeterminada en el CMD básico de Windows).
- **Canal de Errores (STDERR):** El método `startPipeline()` encadena únicamente `STDOUT` con `STDIN`. El canal de errores (`STDERR`) de cada proceso permanece independiente. Si un proceso intermedio falla, se debe revisar su canal de error mediante `getErrorStream()` o configurar `redirectErrorStream(true)`.
- **Cierre de recursos:** Asegurarse de esperar a todos los procesos del pipeline con `.waitFor()` o monitorear sus estados mediante `.onExit()` para evitar la presencia de procesos zombi.

Control Moderno del Proceso con `ProcessHandle` (Java 9+)

Permite obtener métricas del proceso sin acudir a comandos de red o del SO:

```
Java  
import java.io.IOException;  
  
public class EjemploWindows {  
    public static void main(String[] args) throws IOException,  
        InterruptedException {  
        // En Windows se invoca 'timeout' a través de cmd.exe para evitar  
        bloques de entrada interactiva  
        ProcessBuilder pb = new ProcessBuilder("cmd.exe", "/c", "timeout /t 10  
/nobreak");  
        Process p = pb.start();  
  
        // Obtener el identificador de proceso (PID) nativo  
        long pid = p.pid();  
  
        // Obtener información extendida del proceso  
        ProcessHandle.Info info = p.info();  
  
        System.out.println("PID asignado: " + pid);  
        System.out.println("Usuario ejecutor: " +  
            info.user().orElse("Desconocido"));  
    }  
}
```

```
System.out.println("Comando: " + info.command().orElse("N/A"));

// Pequeña pausa de 2 segundos para comprobar su ejecución
Thread.sleep(2000);

// Destrucción limpia o forzada en Windows
if (p.isAlive()) {
    p.destroy(); // En Windows llama a TerminateProcess (cierre
ordenado)
    // p.destroyForcibly(); // Forzado inmediato (no da tiempo a
liberar recursos)
    System.out.println("El proceso fue terminado desde Java.");
}
}
```

Explicación del código paso a paso:

Manejo de excepciones en el método `main`

```
Java
public static void main(String[] args) throws IOException, InterruptedException
```

- Trabajar con el SO implica riesgos externos. `IOException` se lanza si el SO no encuentra el ejecutable o no tiene permisos, e `InterruptedException` salta si el hilo de Java es interrumpido mientras espera a que un proceso termine o mientras duerme.

Configuración del comando nativo en Windows

```
Java
ProcessBuilder pb = new ProcessBuilder("cmd.exe", "/c", "timeout /t 10
/nobreak");
```

- `ProcessBuilder` sirve para **preparar** la orden, no para ejecutarla aún.
- ¿Por qué `cmd.exe /c`? Comandos como `dir` o `timeout` no son ejecutables independientes (`.exe`), sino instrucciones internas del intérprete de Windows. `/c` le dice a la consola: "ejecuta esto y cierra la consola".
- **Argumentos de `timeout`:** `/t 10` hace una pausa de 10 segundos y `/nobreak` ignora si alguien pulsa una tecla en la terminal.

Lanzamiento del subprocesso

Java

```
Process p = pb.start();
```

- **start()** efectúa la llamada al sistema operativo para crear un **proceso hijo**. La ejecución pasa a ser asíncrona: el proceso hijo corre por su cuenta en el SO mientras el programa Java continúa a la siguiente línea.

Inspección de metadatos (Java 9+)

Java

```
long pid = p.pid();  
ProcessHandle.Info info = p.info();  
  
System.out.println("PID asignado: " + pid);  
System.out.println("Usuario ejecutor: " + info.user().orElse("Desconocido"));  
System.out.println("Comando: " + info.command().orElse("N/A"));
```

- **pid()**: Devuelve el identificador único que el SO ha dado a este proceso en la tabla del sistema.
- **info()**: Permite consultar datos del sistema operativo a través de la interfaz `ProcessHandle.Info`.
- **Uso de .orElse(...)**: Muchos métodos de `info` devuelven un `Optional<T>` porque el SO no siempre concede acceso a ciertos datos por privacidad/permisos. `.orElse()` evita que el programa rompa dando un valor por defecto.

Pausa temporal del hilo principal

Java

```
Thread.sleep(2000);
```

- Hacemos que el proceso **padre** (Java) espere 2 segundos. De lo contrario, Java intentaría matar al hijo tan rápido que ni nos daría tiempo a comprobar si el proceso llegó a estar activo en el sistema.

Control de estado y destrucción del subprocesso

Java

```
if (p.isAlive()) {
```

```
p.destroy();  
System.out.println("El proceso fue terminado desde Java.");  
}
```

- **p.isAlive()**: Consulta al SO si el proceso sigue en ejecución (devolverá **true** porque solo han pasado 2 de los 10 segundos de la orden **timeout**).
- **p.destroy()**: Solicita al SO que mate al proceso de forma limpia. En Windows invoca internamente la API **TerminateProcess** liberando la memoria y los recursos asociados.

Gestión de errores y excepciones en la invocación de procesos

Al trabajar con subprocessos en Java, la gestión de errores requiere un tratamiento diferenciado. Un fallo en un subprocesso puede ocurrir a dos niveles completamente distintos:

1. **Excepciones en el hilo de Java** (al intentar crear o interactuar con el proceso).
2. **Errores en la ejecución nativa** (el proceso arranca, pero falla internamente dentro del sistema operativo).

Excepciones Java asociadas a subprocessos

Al invocar o manipular subprocessos, la JVM puede lanzar varias excepciones específicas de **java.lang** o **java.io**:

- **IOException**: Ocurre durante la ejecución de **ProcessBuilder.start()**. Se lanza si el ejecutable no existe, el archivo no tiene permisos de ejecución, o la ruta al comando es incorrecta.
- **InterruptedException**: Ocurre al llamar a **waitFor()**, **waitFor(timeout, unit)** o al leer flujos E/S si el hilo de Java que espera al subprocesso es interrumpido por otro hilo mediante **Thread.interrupt()**.
- **IllegalThreadStateException**: Se lanza si se intenta consultar información que requiere que el proceso haya terminado (como llamar a **process.exitValue()**) mientras el proceso **todavía está en ejecución**.

Captura y aislamiento del flujo de errores nativo (STDERR)

Por defecto, la salida de errores estándar de un subprocesso (**STDERR**) es independiente de su salida de datos (**STDOUT**). Si un comando falla en el sistema operativo, los mensajes de error no aparecerán en **getInputStream()**, sino en **getErrorStream()**.

Existen dos estrategias para capturar y procesar el canal de error:

Lectura independiente del canal de errores (`getErrorStream`)

```
Java
Process proceso = new ProcessBuilder("ls", "/ruta/que/no/existe").start();

// Leer la salida de error del subprocesso
try (BufferedReader errorReader = new BufferedReader(
    new InputStreamReader(proceso.getErrorStream()))) {
    String linea;
    while ((linea = errorReader.readLine()) != null) {
        System.err.println("[STDERR] " + linea);
    }
}
```

Redirección y fusión de flujos (`redirectErrorStream(true)`)

Redirige la salida de errores hacia la salida estándar, mezclando ambos canales en un único flujo de lectura. Evita tener que gestionar dos hilos de lectura independientes.

```
Java
ProcessBuilder pb = new ProcessBuilder("ping",
    "ip_invalida_o_comando_erroneo");
pb.redirectErrorStream(true); // Fusión de STDERR con STDOUT

Process proceso = pb.start();
try (BufferedReader reader = new BufferedReader(
    new InputStreamReader(proceso.getInputStream()))) {
    String linea;
    while ((linea = reader.readLine()) != null) {
        System.out.println("[SALIDA FUSIONADA] " + linea);
    }
}
```

Evaluación del código de salida (*Exit Code*)

Que un subprocesso ejecute sin lanzar una `IOException` en Java **no significa que haya funcionado correctamente**. Se debe comprobar siempre el valor de retorno devuelto por el sistema operativo al terminar el proceso.

- `exitValue() == 0`: Ejecución correcta (Éxito).
- `exitValue() != 0`: Error de ejecución en el sistema operativo.

```
Java
int exitCode = proceso.waitFor();
if (exitCode != 0) {
    System.err.println("El proceso finalizó con un código de error: " +
exitCode);
}
```

Ejemplo Completo: Patrón de Robustez y Control de Errores

El siguiente código resume la forma profesional y segura de invocar un proceso externo en Java, controlando todas las excepciones de la JVM, aislando los canales de error y previniendo bloqueos (*deadlocks*).

```
Java
import java.io.BufferedReader;
import java.io.File;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.concurrent.TimeUnit;

public class GestionErroresProcesos {

    public static void main(String[] args) {
        // Comando que provocará un error intencionado (archivo inexistente)
        ProcessBuilder pb = new ProcessBuilder("cat",
"archivo_inexistente.txt");

        // Opcional: Redirigir la salida de error a un fichero de log en disco
        // pb.redirectError(new File("errores.log"));

        Process proceso = null;
        try {
            System.out.println("Iniciando subprocesso...");
            proceso = pb.start();

            // Capturar la salida estándar de errores (STDERR)
            StringBuilder erroresBuffer = new StringBuilder();
            try (BufferedReader errorReader = new BufferedReader(
                new InputStreamReader(proceso.getErrorStream()))) {
                String linea;
                while ((linea = errorReader.readLine()) != null) {
                    erroresBuffer.append(linea).append("\n");
                }
            }
        }
    }
}
```

```
// Esperar al proceso con un tiempo máximo de seguridad
boolean finalizado = proceso.waitFor(5, TimeUnit.SECONDS);

if (!finalizado) {
    System.err.println("TIMEOUT: El proceso superó el tiempo
máximo. Forzando cierre...");
    proceso.destroyForcibly();
} else {
    int exitCode = proceso.exitValue();
    if (exitCode == 0) {
        System.out.println("El proceso se ejecutó correctamente.");
    } else {
        System.err.println("ERROR NATIVO (Código " + exitCode +
");");
        System.err.println(erroresBuffer.toString());
    }
}

} catch (IOException e) {
    // Error al lanzar el comando (no existe el comando, sin permisos,
etc.)
    System.err.println("EXCEPCIÓN E/S: No se pudo lanzar el subprocesso
-> " + e.getMessage());
} catch (InterruptedException e) {
    // El hilo principal de Java fue interrumpido durante la espera
    System.err.println("EXCEPCIÓN INTERRUPCIÓN: El hilo principal fue
interrumpido -> " + e.getMessage());
    Thread.currentThread().interrupt(); // Restablecer estado de
interrupción
} catch (IllegalThreadStateException e) {
    // Intento ilegal de consultar el estado de un proceso que no ha
terminado
    System.err.println("EXCEPCIÓN ESTADO: Intento de acceso a un
proceso en ejecución -> " + e.getMessage());
} finally {
    // Asegurar limpieza de recursos si el proceso sigue vivo por algún
motivo
    if (proceso != null && proceso.isAlive()) {
        proceso.destroyForcibly();
    }
}
}
```

Ejercicio Práctico: Monitor de Diagnóstico y Generador de Logs

Enunciado

Crea un programa en Java que ejecute un comando de diagnóstico de red (`ping` a `localhost`). El programa debe capturar los metadatos del proceso hijo por pantalla y guardar todo el resultado de la ejecución del comando en un archivo de log llamado `registro_ping.log`.

Objetivos del ejercicio

- Configurar y lanzar subprocessos con `ProcessBuilder`.
- Redirigir salidas del proceso directamente a archivos del sistema sin saturar la memoria RAM.
- Consultar metadatos del proceso en tiempo de ejecución (`PID` y estado).
- Gestionar correctamente los códigos de salida (`exit values`) y excepciones.

Requisitos paso a paso

1. Detectar el sistema operativo para ejecutar `ping 127.0.0.1` con los argumentos correctos (`-n 4` en Windows o `-c 4` en Linux/macOS).
2. Redirigir la salida estándar del proceso (`STDOUT`) directamente al fichero `registro_ping.log`.
3. Iniciar el proceso y mostrar por la consola de Java:
4. El PID asignado al proceso hijo.
5. Si el proceso está activo en ese instante.
6. Esperar a que el proceso hijo finalice su tarea.
7. Imprimir en la consola el código de finalización (`0` = OK, distinto de `0` = Error) y confirmar al usuario que el archivo de log ha sido generado.

Ampliación:

- Modificar el programa para que en lugar de sobrescribir el archivo `registro_ping.log`, **añada** información al final del archivo existente usando `Redirect.appendTo(logFile)`.
- Añadir al inicio del fichero de log una cabecera con la fecha y hora exacta de la ejecución mediante Java.

Bibliografía

Libros de texto recomendados:

- Programación de Servicios y Procesos, *Fernando Paniagua Martín* - Editorial Paraninfo
- Programación de Servicios y Procesos, *Carlos Alberto Cortijo Bon* - Editorial Síntesis